

HEALTH NORTH

Protocole de Tests API

Dossier 3 — Mission 3

Projet	Health North — Application de réservation médicale
Dossier	Dossier 3 — Backoffice / API
Mission	Mission 3 — Protocole de tests API
Outil de test	Bruno v3.1.4 (collection versionnée Git)
Environnement	Production — https://healthnorth.onrender.com
Date d'exécution	15 mars 2026

1. Contexte et objectifs

Ce document constitue le protocole de tests de l'API JSON de l'application Health North. Il couvre deux types d'endpoints : les endpoints de lecture dynamique utilisés par le formulaire de prise de rendez-vous, et les endpoints CRUD complets pour la gestion des rendez-vous.

1.1 Rôle de l'API dans l'application

Health North expose deux catégories d'endpoints :

- Endpoints de lecture (GET) — alimentent les listes déroulantes en cascade du formulaire de prise de RDV : villes par région, cliniques par ville, spécialistes par clinique, examens par spécialiste
- Endpoints CRUD — permettent la gestion complète des rendez-vous via API : lecture, création, modification, annulation

1.2 Choix de l'outil : Bruno

Bruno a été retenu comme outil de tests API car ses fichiers de collection sont en texte brut (.bru) et se committent dans Git, démontrant une démarche professionnelle de gestion des tests versionnés.

1.3 Sécurité et authentification

Tous les endpoints API sont protégés par le décorateur `@login_required` de Django. Le protocole inclut une étape d'authentification préalable. Bruno gère automatiquement les cookies de session une fois la connexion établie.

2. Endpoints testés

#	Méthode	URL	Type	Catégorie
1	GET	/appointments/api/cities/{region}/	Lecture	Formulaire
2	GET	/appointments/api/clinics/{city}/	Lecture	Formulaire
3	GET	/appointments/api/specialists/{id}/	Lecture	Formulaire
4	GET	/appointments/api/exam-types/{id}/	Lecture	Formulaire
5	GET	/appointments/api/appointments/	Read	CRUD
6	POST	/appointments/api/appointments/	Create	CRUD
7	PUT	/appointments/api/appointments/{id}/	Update	CRUD
8	DELETE	/appointments/api/appointments/{id}/	Delete	CRUD

3. Procédure d'authentification

L'authentification Django repose sur deux mécanismes : le token CSRF et le cookie de session.

3.1 Étape 1 — Récupération du token CSRF

Champ	Valeur
Méthode	GET
URL	{{baseUrl}}/accounts/login/
Réponse attendue	200 OK + Set-Cookie: csrftoken=...

3.2 Étape 2 — Connexion

Champ	Valeur
Méthode	POST
URL	{{baseUrl}}/accounts/login/
Header X-CSRFToken	{{csrfToken}}
Body (form)	username=patient1 / password=NewPass456!
Réponse attendue	200 OK — redirection vers la page d'accueil

4. Résultats — Endpoints de lecture (formulaire)

4.1 Tableau de synthèse

Endpoint	Assertion	Attendu	Obtenu	Résultat
GET villes par région	res.status eq 200	200	200	✓ PASS
GET villes par région	res.body[0] eq Paris	Paris	Paris	✓ PASS
GET cliniques par ville	res.status eq 200	200	200	✓ PASS
GET cliniques par ville	res.body[0].name eq HN Paris Centre	HN Paris Centre	HN Paris Centre	✓ PASS
GET spécialistes	res.status eq 200	200	200	✓ PASS
GET spécialistes	res.body[0].name eq Dr. Dubois	Dr. Dubois	Dr. Dubois	✓ PASS
GET examens	res.status eq 200	200	200	✓ PASS
GET examens	res.body[0].name eq Scanner thoracique	Scanner thor.	Scanner thor.	✓ PASS

4.2 Test 1 — GET villes par région

URL testée : `GET {{baseUrl}}/appointments/api/cities/ile-de-france/`

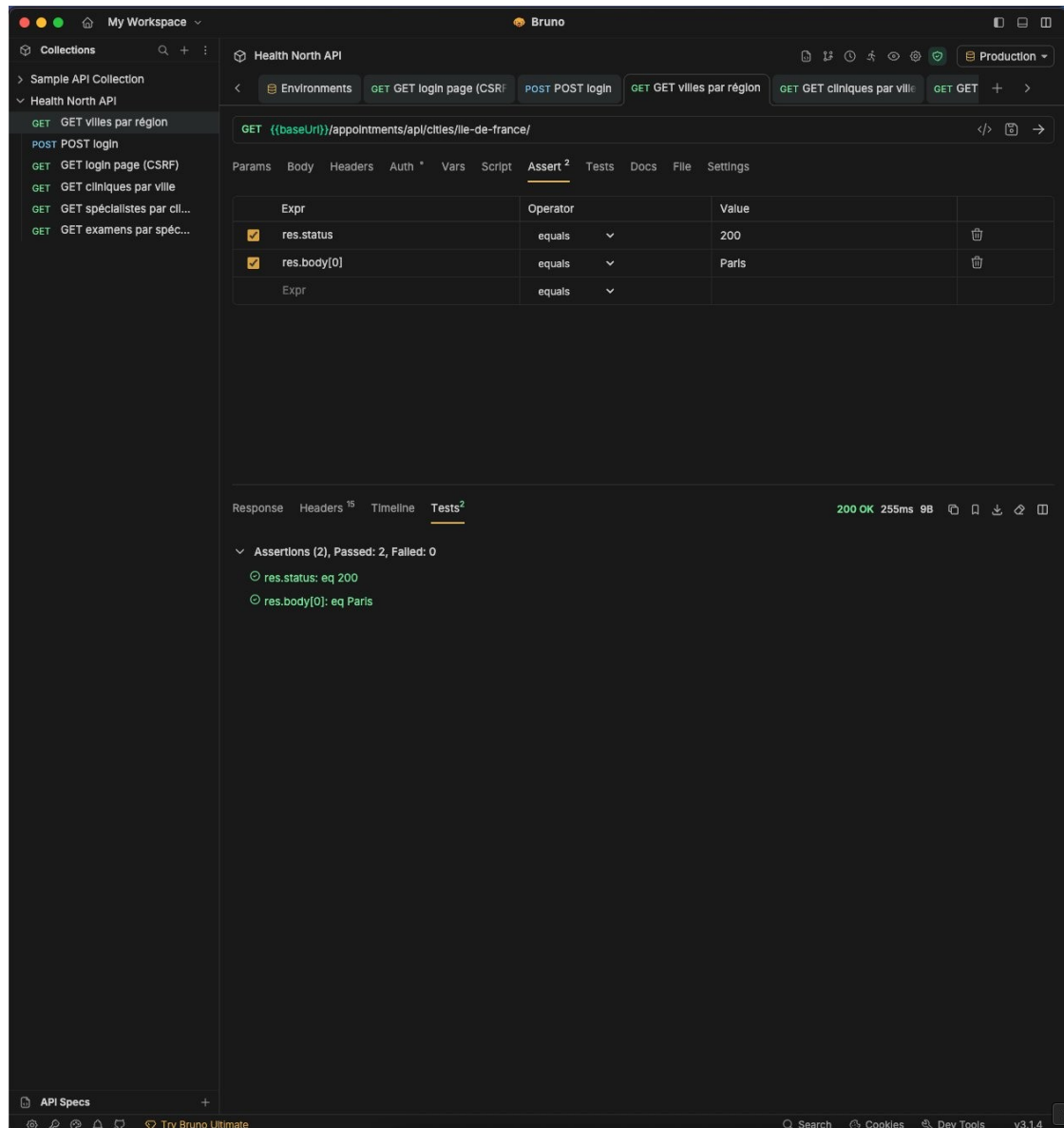


Figure 1 — GET villes par région : 2 assertions passées

4.3 Test 2 — GET cliniques par ville

URL testée : `GET {{baseUrl}}/appointments/apl/clinics/Paris/`

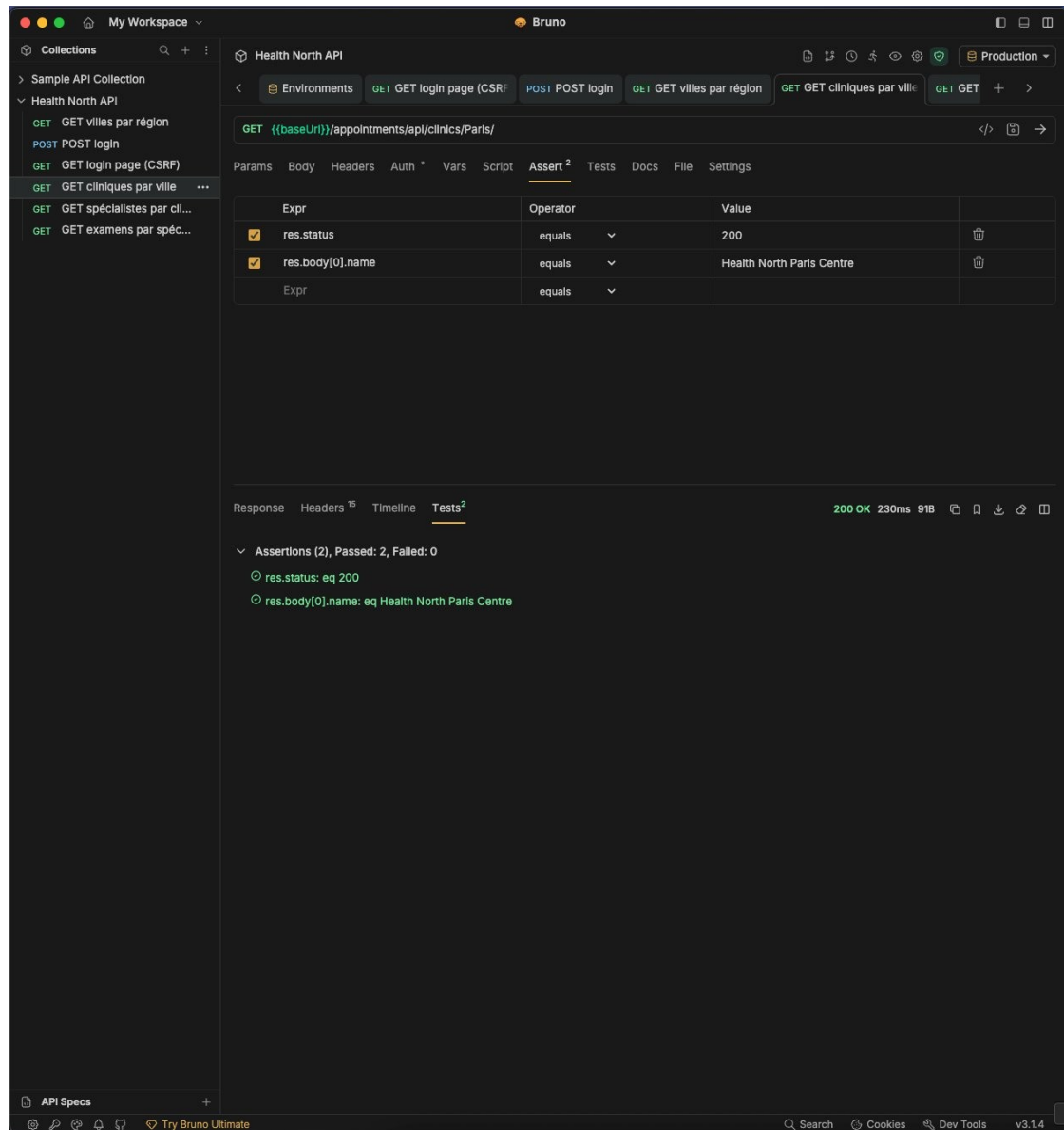


Figure 2 — GET cliniques par ville : 2 assertions passées

4.4 Test 3 — GET spécialistes par clinique

URL testée : `GET {{baseUrl}}/appointments/api/specialists/1/`

The screenshot shows the Bruno API client interface. The left sidebar displays a collection named 'Health North API' with several endpoints. The main panel shows a GET request to `{{baseUrl}}/appointments/api/specialists/1/`. The 'Assert' tab is active, displaying a table of assertions:

Expr	Operator	Value
☒ <code>res.status</code>	<code>equals</code>	<code>200</code>
☒ <code>res.body[0].name</code>	<code>equals</code>	<code>Dr. Dubois</code>
<code>Expr</code>	<code>equals</code>	

Below the assertions, the 'Tests' tab shows the results: 'Assertions (2), Passed: 2, Failed: 0'. The specific assertions are listed as:

- `res.status: eq 200`
- `res.body[0].name: eq Dr. Dubois`

The status bar at the bottom indicates a successful response: `200 OK 42.55s 121B`.

Figure 3 — GET spécialistes : 2 assertions passées

4.5 Test 4 — GET examens par spécialiste

URL testée : `GET {{baseUrl}}/appointments/api/exam-types/1/`

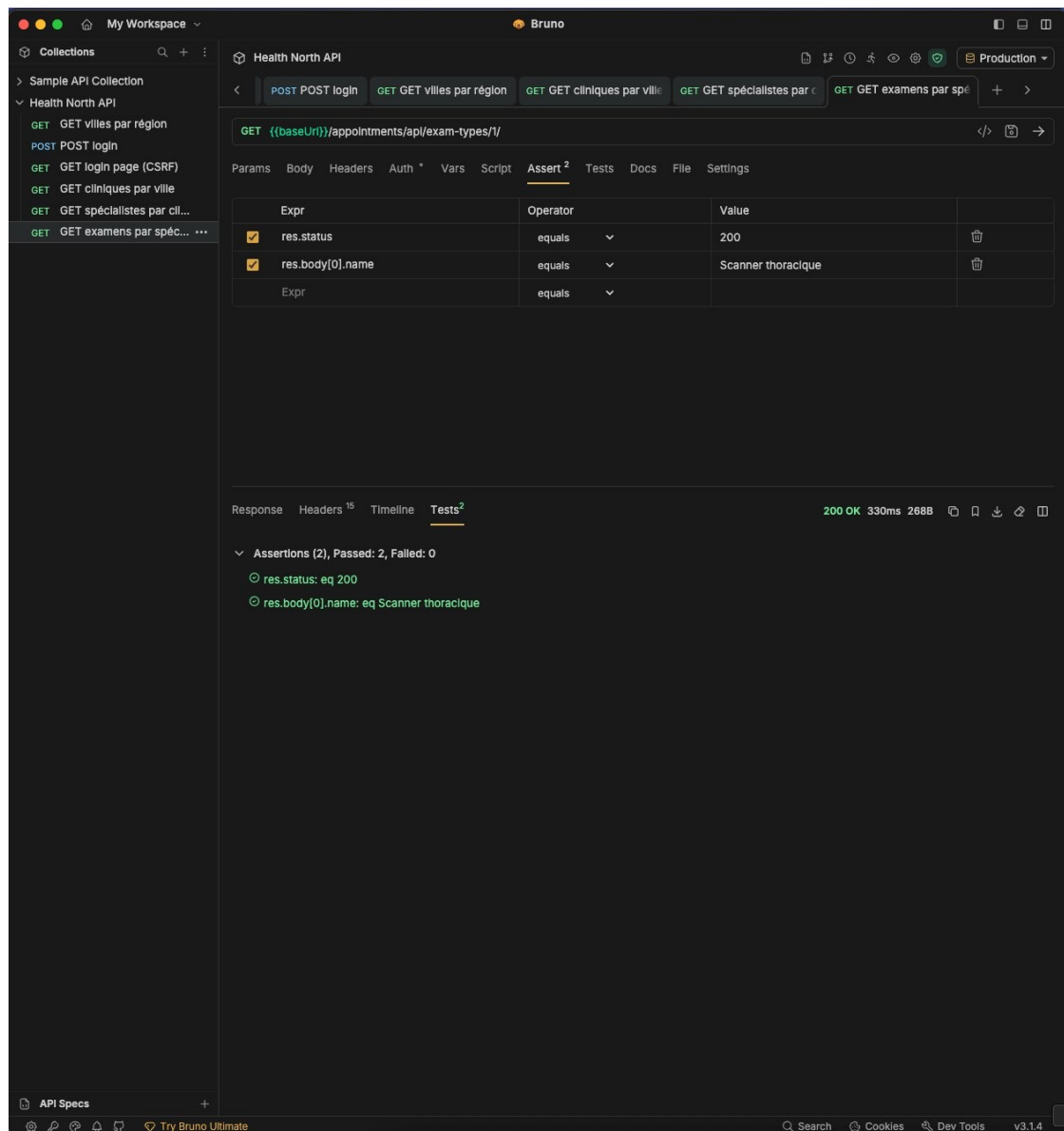


Figure 4 — GET examens : 2 assertions passées

5. Résultats — Endpoints CRUD

Les endpoints CRUD permettent la gestion complète des rendez-vous via API. Ils sont utilisés notamment par l'application mobile (maquettée dans le Dossier 1 Mission 1) et démontrent la conformité avec l'exigence de l'énoncé : "Implémentez un service d'API permettant d'obtenir des accès CRUD sur les bases principales de votre application."

5.1 Tableau de synthèse CRUD

Endpoint	Assertion	Attendu	Obtenu	Résultat
GET liste appointments	res.status eq 200	200	200	✓ PASS
GET liste appointments	res.body[0].specialist eq Dr. Martin	Dr. Martin	Dr. Martin	✓ PASS
POST créer appointment	res.status eq 201	201	201	✓ PASS
POST créer appointment	res.body.status eq created	created	created	✓ PASS
PUT modifier appointment	res.status eq 200	200	200	✓ PASS
PUT modifier appointment	res.body.status eq updated	updated	updated	✓ PASS
DELETE annuler appointment	res.status eq 200	200	200	✓ PASS
DELETE annuler appointment	res.body.status eq cancelled	cancelled	cancelled	✓ PASS

Bilan CRUD : 8 assertions sur 8 — 100% de réussite.

5.2 Test 5 — GET liste appointments

URL testée : `GET {{baseUrl}}/appointments/api/appointments/`

Retourne la liste de tous les rendez-vous du patient connecté avec les informations complètes (spécialiste, spécialité, clinique, examen, statut).

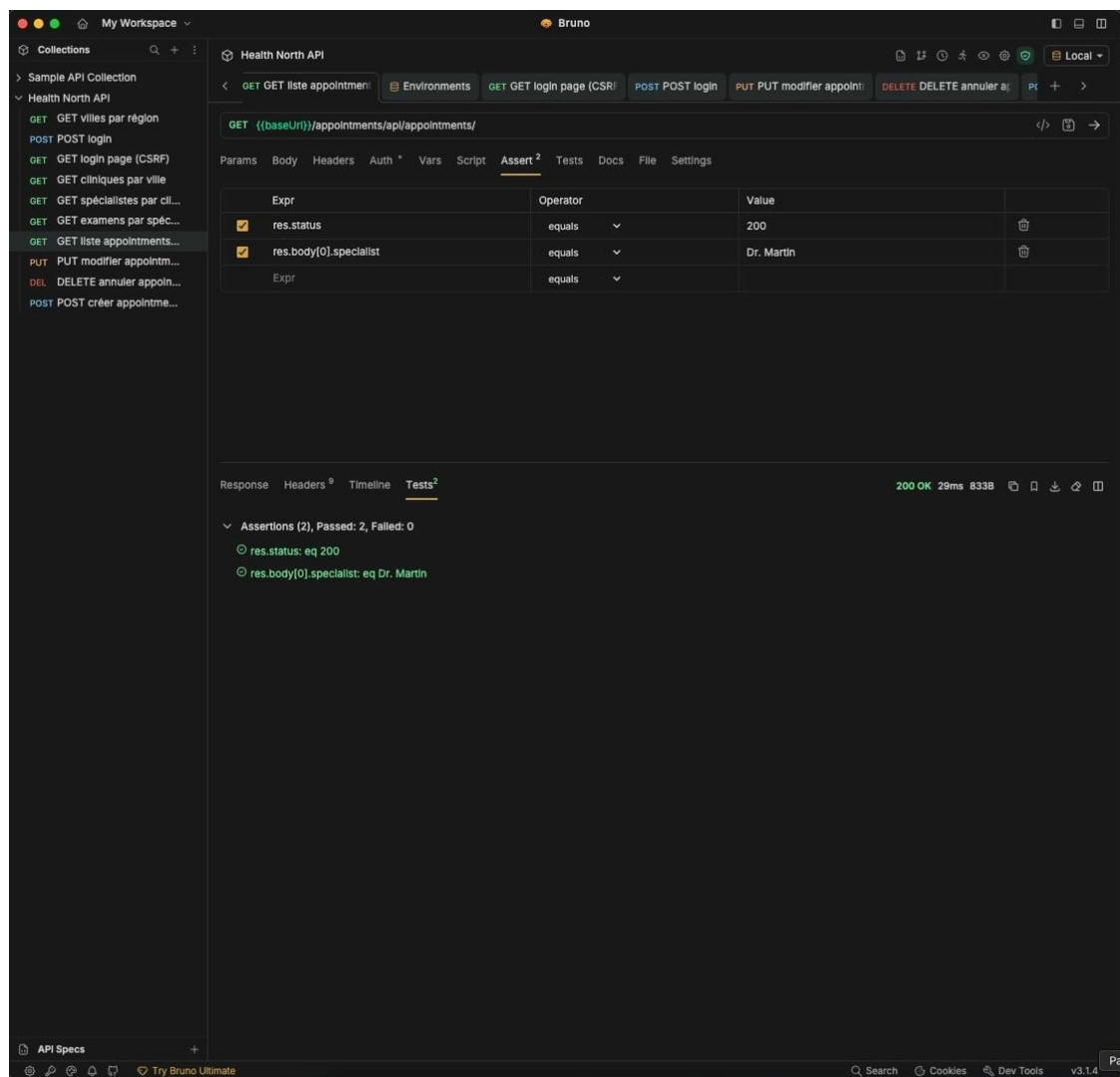


Figure 5 — GET liste appointments : 2 assertions passées

5.3 Test 6 — POST créer appointment

URL testée : `POST {{baseUrl}}/appointments/api/appointments/`

Crée un nouveau rendez-vous en envoyant les données au format JSON (specialist_id, clinic_id, exam_type_id, date, notes). Retourne l'id du rendez-vous créé et le statut 'created'.

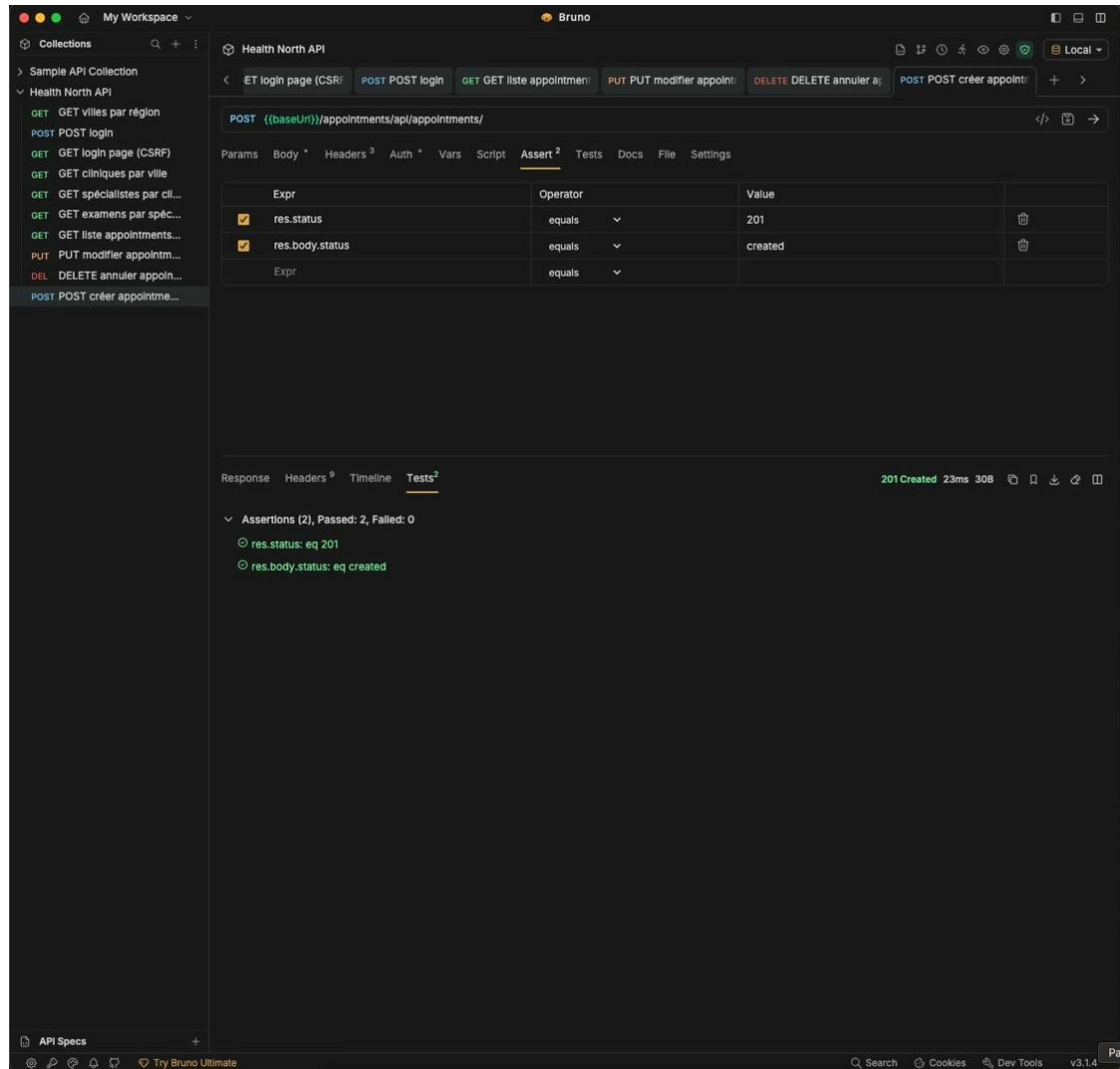


Figure 6 — POST créer appointment : statut 201 Created

5.4 Test 7 — PUT modifier appointment

URL testée : `PUT {{baseUrl}}/appointments/api/appointments/{id}/`

Modifie un rendez-vous existant (date, notes, statut). Retourne l'id et le statut 'updated'.

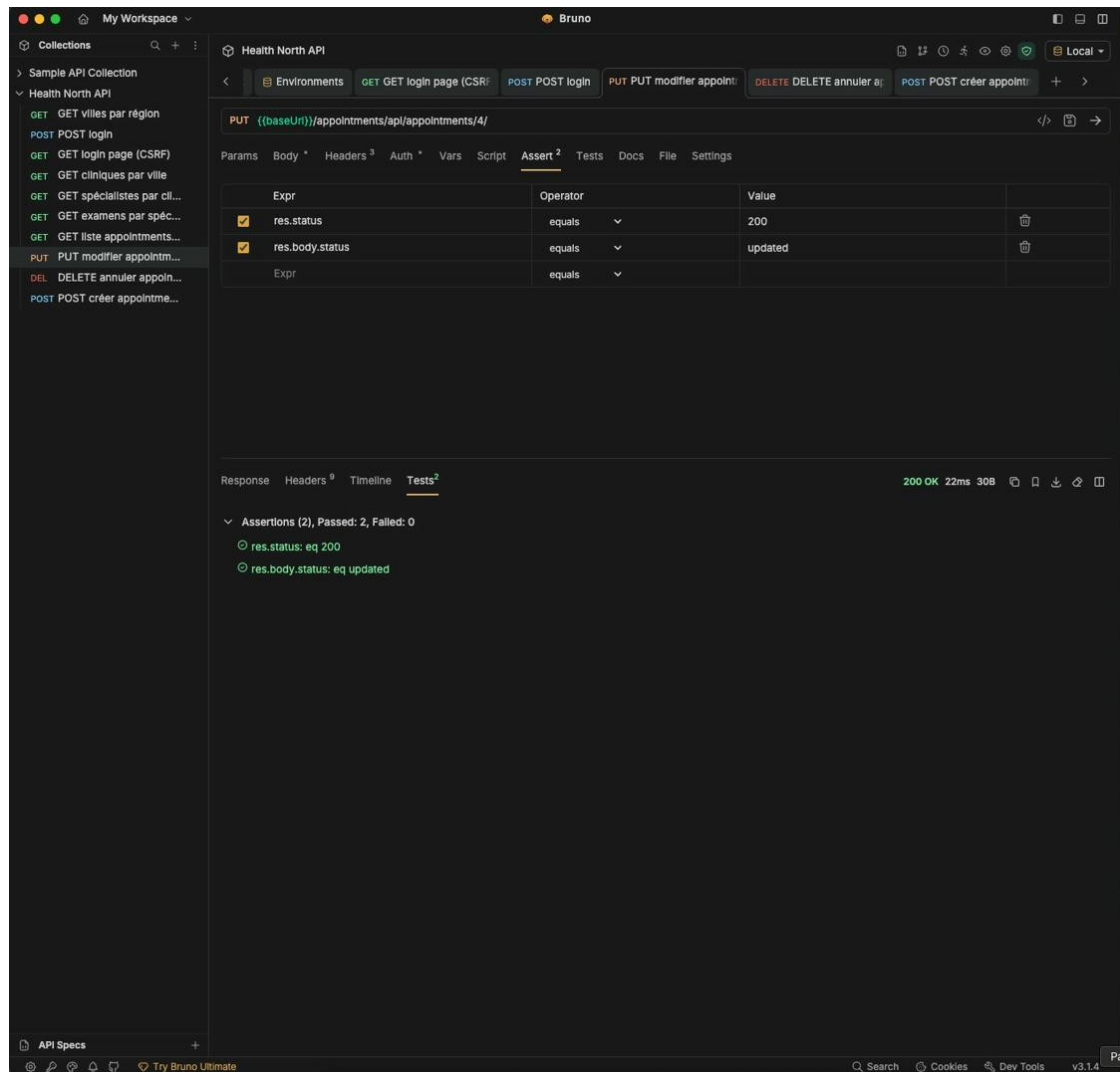


Figure 7 — PUT modifier appointment : 2 assertions passées

5.5 Test 8 — DELETE annuler appointment

URL testée : `DELETE {{baseUrl}}/appointments/api/appointments/{id}/`

Annule un rendez-vous (soft delete — passage en statut 'cancelled'). Retourne l'id et le statut 'cancelled'. Le rendez-vous reste en base pour préserver l'historique du patient.

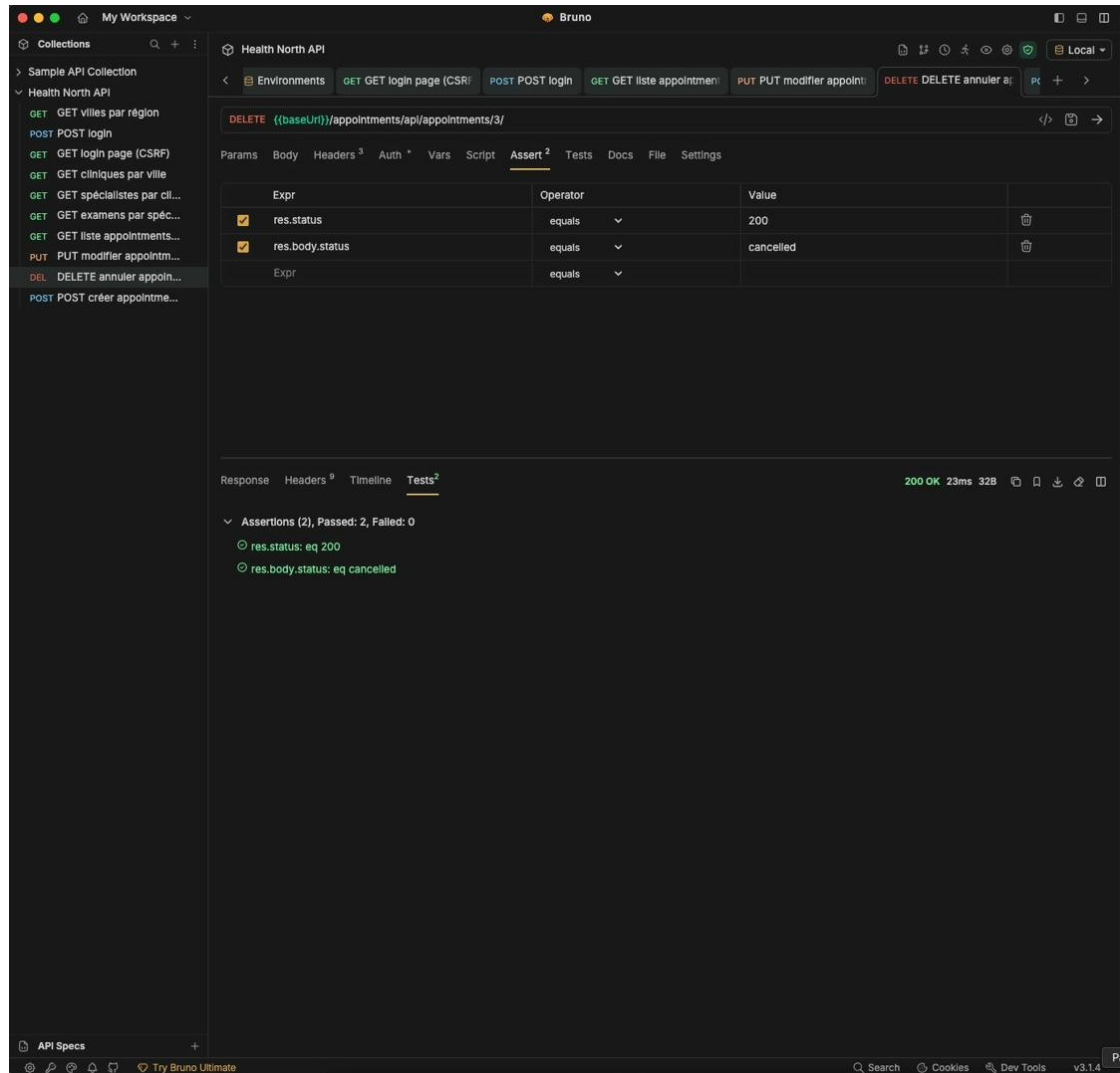


Figure 8 — DELETE annuler appointment : 2 assertions passées

6. Collection Bruno versionnée

La collection Bruno est versionnée dans le repository GitHub sous le dossier bruno/. Elle contient 9 requêtes (authentification + 4 endpoints lecture + 4 endpoints CRUD) avec assertions automatisées.

Fichier	Contenu
GET login page (CSRF).bru	Récupération du token CSRF
POST login.bru	Authentification
GET villes par région.bru	Test endpoint /api/cities/
GET cliniques par ville.bru	Test endpoint /api/clinics/
GET spécialistes par clinique.bru	Test endpoint /api/specialists/
GET examens par spécialiste.bru	Test endpoint /api/exam-types/
GET liste appointments (CRUD).bru	CRUD Read
POST créer appointment (CRUD).bru	CRUD Create
PUT modifier appointment (CRUD).bru	CRUD Update
DELETE annuler appointment (CRUD).bru	CRUD Delete

7. Conclusion

Le protocole de tests API de Health North couvre l'intégralité des 8 endpoints exposés par l'application. Les 16 assertions définies (8 pour les endpoints de lecture + 8 pour le CRUD) ont toutes passé avec succès.

La collection Bruno versionnée dans Git constitue un livrable technique reproductible : n'importe quel développeur peut cloner le repository, ouvrir Bruno, et relancer l'ensemble des tests pour vérifier que l'API fonctionne correctement après une modification du code.

Ce protocole démontre la maîtrise des bonnes pratiques de test d'API : authentification sécurisée, assertions automatisées, couverture CRUD complète, et versionning des tests avec le code source.